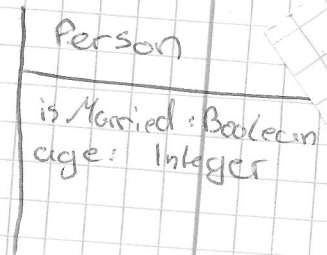


context <classifier>

<constraintType>[<constraintName>]; <bool-exp>

<Classifier>: 1 Klassen - /Methodenname

<constraintType>: inv, pre, post



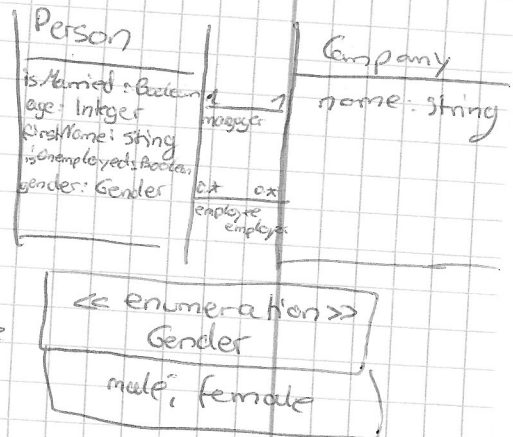
context Person

inv VerheiratetesIndividuell: self.isMarried ^{implies} self.age >= 18
oder
self.isMarried implies (self.age >= 18)
Vergleichsoper.

Punkt bei einzelnen Objekten
Pfeile bei Collections

context Company
inv: self.employee → size() > 3
↑
Object Collection

infix: >=, <=, <, >, <> =
and, or, xor, not, implies



context Person

inv: self.firstName = 'Susan' implies
self.gender = Gender::female

Funktionen auf Collections: ~~sum()~~, size

primitive Typen: Real, Integer,
String, Boolean

sum(): ~~Integer~~ Real

isEmpty(): Boolean

~~count()~~:

includes(o: T): Boolean excludes(o: T): Boolean

includesAll(c: Collection(T)): Boolean

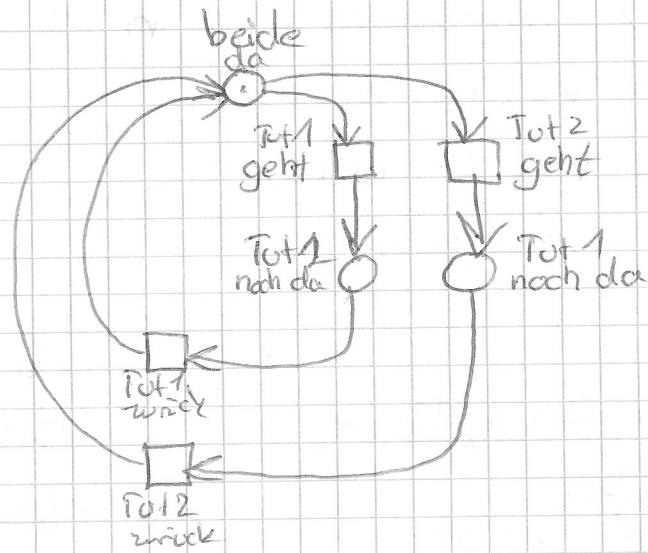
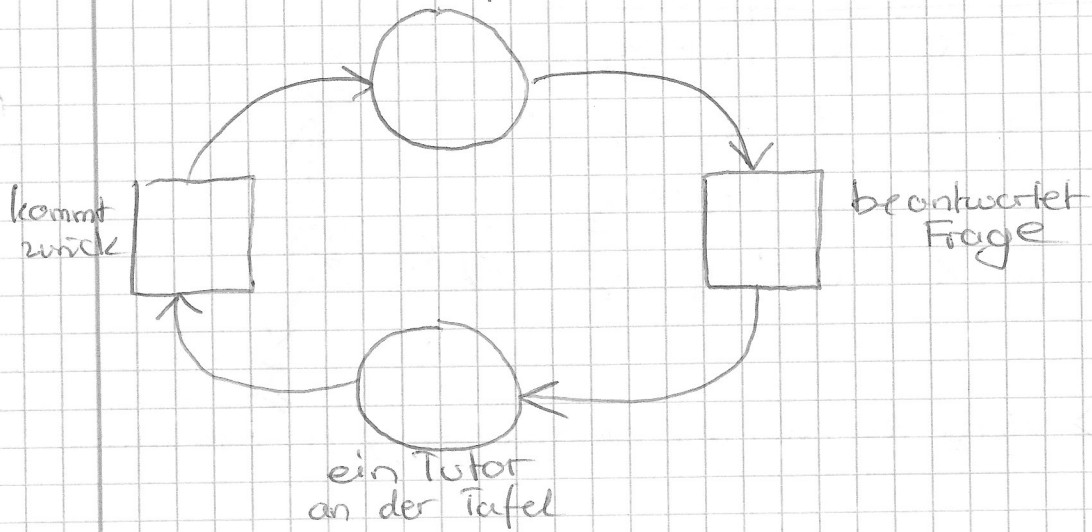
size(): Integer

notEmpty: Boolean

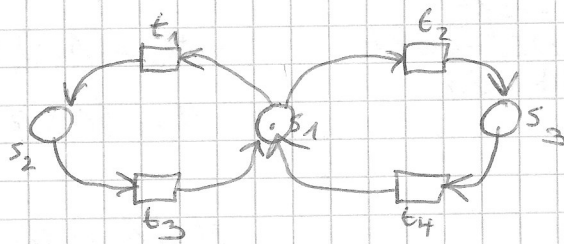
count(o: T): Integer

beide Tutoren
an der Tafel

SWK Blatt 2



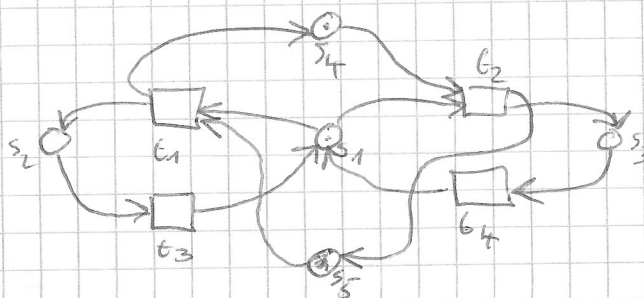
2.1)



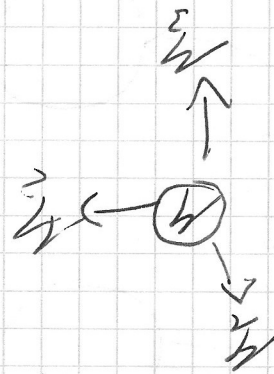
2)

i	s ₁	s ₂	s ₃	$t \rightarrow M_i$
M ₀	1	0	0	$t_1 \rightarrow M_1, t_2 \rightarrow M_2$
M ₂	0	1	0	$t_3 \rightarrow M_0$
M ₂	0	0	1	$t_4 \rightarrow M_0$

3)



i	s ₁	s ₂	s ₃	s ₄	t → M _i
M ₀	1	0	0	0	t ₁ → M ₁
M ₁	0	1	0	1	t ₃ → M ₂
M ₂	1	0	0	1	t ₂ → M ₃
M ₃	0	0	1	0	t ₄ → M ₀

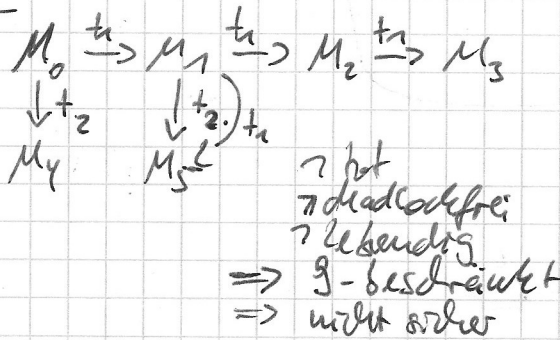


$[M_0]$ Menge der von der Startmarkierung M_0 erreichbaren Markierungen

$M[t]$: Transition t ist in Markierung M aktiviert

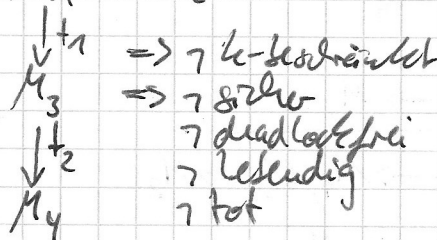
$$M[t] \Leftrightarrow \forall s \in t: M(s) \geq 1 \quad \underbrace{\forall s \in t: W(L, s)}_{\text{Nachbereich von } t}$$

2.4.2.

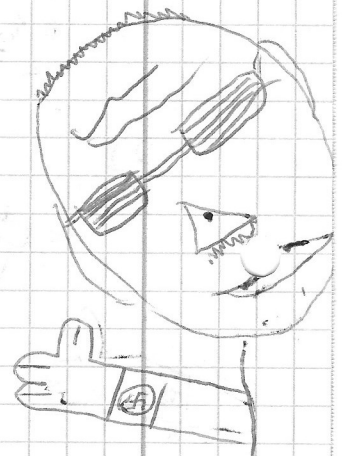


$$\begin{aligned} M_0 &= (3, 0, 1, 0) \\ M_1 &= (2, 1, 1, 0) \\ M_2 &= (1, 6, 1, 0) \\ M_3 &= (0, 3, 1, 0) \\ M_4 &= (1, 0, 0, 1) \\ M_5 &= (0, 3, 0, 1) \end{aligned}$$

2.4.3. $M_0 \xrightarrow{t_2} M_1 \xrightarrow{t_3} M_2$

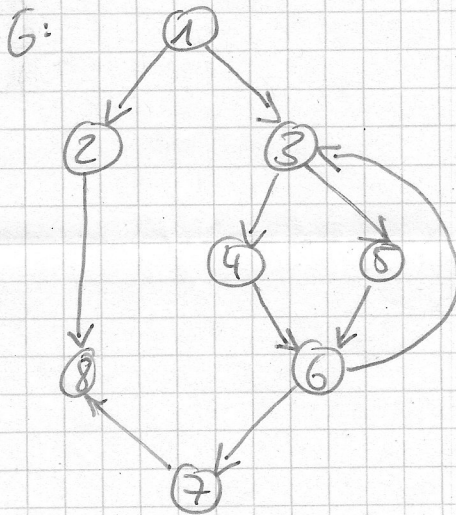


$$\begin{aligned} M_0 &= (1, 0, 0) \\ M_1 &= (0, 1, 1) \\ M_2 &= (0, 0, 0) \\ M_3 &= (1, 0, 1) \\ M_4 &= (0, 1, 2) \end{aligned}$$



SWK-Blatt 4

Aufgabe 4.5



$$r(G) = e - v + 2$$

$$= 10 - 8 + 2$$

$$= 4$$

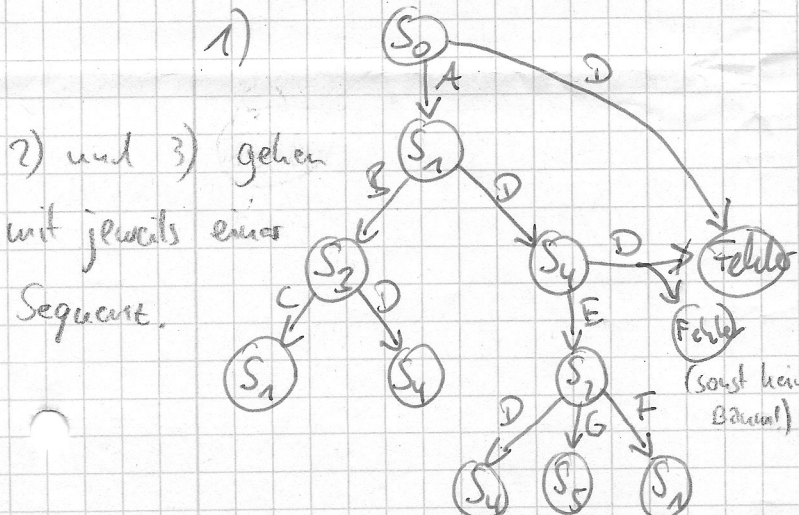
Gr. 7

Aufgabe 4.6

Klasse	WMC	DIT	NOC
A	2	0	1
B	1	1	2
C	0	2	12
D	1	3	0
E	2	3	0

$$C(i) = 1$$

Aufgabe 4.8



- 2) $\langle S_0 \rangle A \langle S_1 \rangle B \langle S_2 \rangle$
 $\langle S_0 \rangle A \langle S_1 \rangle D \langle S_4 \rangle E \langle S_2 \rangle G \langle S_5 \rangle$
- 3) $\langle S_0 \rangle A \langle S_1 \rangle B \langle S_2 \rangle C \langle S_1 \rangle$ $S_2 \rightarrow S_4$ fehlt
 $\langle S_0 \rangle A \langle S_1 \rangle D \langle S_4 \rangle E \langle S_2 \rangle F \langle S_1 \rangle$ $S_3 \rightarrow S_4$ fehlt
 $\langle S_0 \rangle A \langle S_1 \rangle D \langle S_4 \rangle E \langle S_2 \rangle G \langle S_5 \rangle$
- 4) $\langle S_0 \rangle D \langle \text{Fehler} \rangle$
- 5) $\langle S_0 \rangle A \langle S_1 \rangle D \langle S_4 \rangle D \langle \text{Fehler} \rangle$

SWV)

4.1.1) a) = Sequenz 1

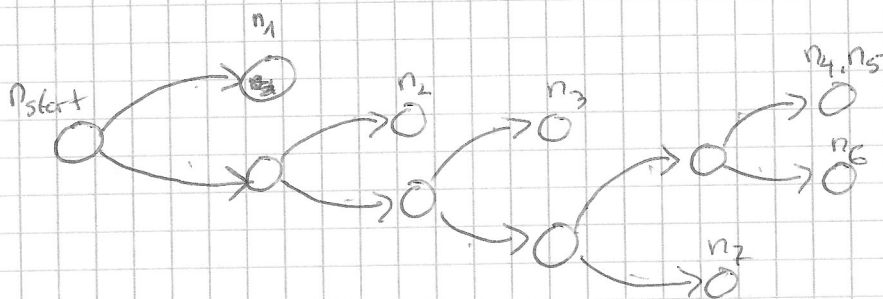
b) if ohne else 2

c) if mit else 2

d) Fallunterscheidung mit 3-Fällen 3

e) Schleife 2

Anzahl Kanten - Anzahl Knoten
+ 2



Knoten: 7

Kanten: 10

$$K \equiv \text{Kanten} - \text{Knoten} + \text{Endknoten} + 1$$

UUCW = wie viele Transaktionen

Anzahl Transaktionen Gewicht = 5, 10, ~~50~~ 15

$$U_1, U_2 = 5 \quad (1-3)$$

$$U_3 = 10 \quad (3-7)$$

$$U_4, U_5 = 15 \quad (8-\infty)$$

$$UUCW = \sum_{i=0} U_i = 50$$

$$UAW = 6$$

$$TF = 19,5$$

$$TCF = 0,195 + 0,6 = 0,795$$

$$EF =$$

$$ECF = 1,445$$

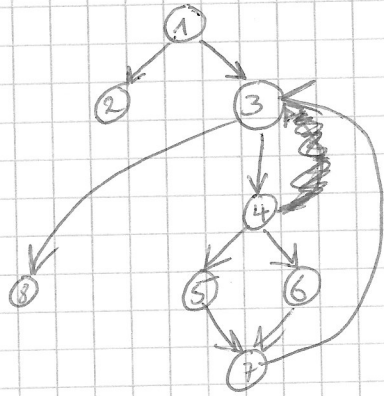
$$UCP = (UUCW + UAW) \cdot TCF \cdot ECF$$

$$= 64,3314$$

$$\approx 1415 \text{ Stunden}$$

4.5

$G =$



$$v(G) = 10 - 8 + 2 = 4$$

f) e) b) g) i) a) d) i) h) c)

Sopra

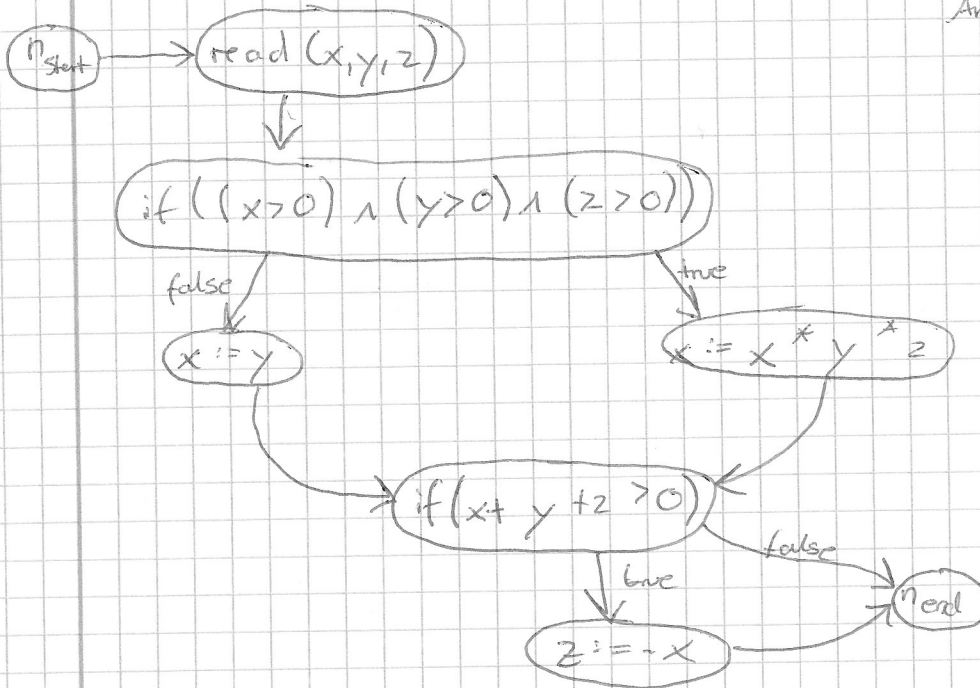
- noch cancel Zug neu beginnen
- Aktivität vor der Route

Startbildschirm

5.1)

1) jede Anweisung min. einmal aufrufen

Anweisung sind nur
Zuweisungen



1. $x = 1$ $y = 1$ $z = 1$
 $x = 0$ $y = 0$ $z = 0$
 $x =$

5.3.2 Release management

1. Commit stage
2. Acceptance test stage
3. Manual test stage
4. Performance test stage
5. Release stage

5.2	5.3.1	5.3.2	5.3.3	Σ
14	6	/	6	26

- zu 1) Hier wird Code generiert, dieser wird dann durch ~~den~~ Unit-Tests getestet und noch in ein Format gepackt
- zu 2) Hier werden reale Anwendungsfälle getestet
- zu 3) explorative Tests
- zu 4) Hier wird die Performance getestet.
im Beispiel: Performanz, Durchsatz, Verfügbarkeit...
- zu 5) Hier wird das Programm auf einem System installiert oder als installierbare Datei exportiert

2) a) Blue-Green-Deployment:

Auf den Geräten befindet sich eine alte Version des Programmes. Dies sei jetzt mal die blaue. Nun wird die grüne auf alle Produktumgebungen aufgespielt. Nun kann die grüne Umgebung verändert und getestet werden. Ist sie fehlerfrei, so kann der Router blitzschnell zwischen der grünen und der blauen umstellen und so die aktuelle Umgebung feststellen/festlegen. Ein weiterer Vorteil ist, dass man direkt auf eine funktionierende Version zurückstellen kann.

Kanarienvogel Release:

Hier wird die neue Version auf eine kleine Testmenge ^{gerichtet} ~~geschickt~~ ~~geschickt~~. Diese Testgruppe kann nun diese Version testen. Treten hier Fehler auf, so kann die Version auch bei diesen System schnell zurückgesetzt werden.

b) Beide Versionen werden ~~von~~ auf (eingesetzte) Systemen implementiert. Nach den Tests wird diese Route dann (auf die User) umgeschaltet.

Usersysteme werden mit der neuen Version upgegradet. Es wird nur in der Anzahl unterschieden.

Beide Ansätze unterstützen zurückschalten auf alte Version.

Aufgabe 5.3.1

- 1) 1. Harri sperrt Datei. (Erfolgreich, weil Datei noch nicht gesperrt)
2. Harri liest Datei.
3. Harri schreibt in die Datei.
4. Harri entsperrt Datei. die Datei starkste würde
Wenn Salli zwischen 1. und 4., so ist ihr das nicht möglich.
5. Salli sperrt Datei.
;

- 2) - vergessen zu entsperren
- kein paralleles Arbeiten möglich
- Deadlocks möglich.

5.3.2 fehlt

Aufgabe 6.3

$A = \{$

$\text{real} \stackrel{\text{def}}{=} \mathbb{R}$

$\text{zero} \stackrel{\text{def}}{=} 0$

$\text{one} \stackrel{\text{def}}{=} 1$

$\forall u, m \in \mathbb{R}. \text{add}(u, m) \stackrel{\text{def}}{=} u + m$

$\forall u, m \in \mathbb{R}. \text{sub}(u, m) \stackrel{\text{def}}{=} u - m$

$\forall u, m \in \mathbb{R}. \text{mult}(u, m) \stackrel{\text{def}}{=} u \cdot m$

$\forall u, m \in \mathbb{R} \ m \neq 0 \ \text{div}(u, m) \stackrel{\text{def}}{=} \frac{u}{m}$

$\}$

Aufgabe 6.4

algebra stack introduces sorts object, nat, stack
operations

$\text{empty} : \rightarrow \text{stack}$

$\text{push} : \text{stack}, \text{object} \rightarrow \text{stack}$

$\text{pop} : \text{stack} \rightarrow \text{stack}$

$\text{top} : \text{stack} \rightarrow \text{object}$

$\text{size} : \text{stack} \rightarrow \text{nat}$

constraints $\text{empty}, \text{push}, \text{pop}, \text{top}, \text{size}$

so that for all $st : \text{stack}, o : \text{object}$

$\text{size}(\text{empty}) = 0$

$\text{size}(\text{push}(st, o)) = \text{size}(st) + 1$

$\text{pop}(\text{empty}()) = \text{empty}()$

$\text{pop}(\text{push}(st, o)) = st$

$\text{top}(\text{push}(st, o)) = o$

$\text{top}(\text{empty}()) = \text{error}$

(7)

Julius Jacobson 166132

T.U. Schallau 165411

Cedric Schürer 166215

Jonas Schmidt 166186

6..	3	4	5	6	7	8	9	10	Σ
	2	6	4	4	2	/	7	8	32

$0, +, 1$ müssen in Signatur
definiert werden

Aufgabe 6.5

1. Eine algebraische Spezifikation ist redundant, wenn die Menge der beweisbaren Aussagen gleich bleibt; auch dann, wenn ein Axiom gestrichen wird.
2. Eine algebraische Spezifikation ist widersprüchlich, wenn ~~es~~^{Widerspruch} $(true = false)$ bewiesen lässt.
3. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip no-junk, wenn jedes Element des Modells mittels Termen der Signatur bezeichnet werden kann.
4. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip no-confusion, wenn in dem Modell keine Gleichungen gelten, die nicht zwingend aus den Axiomen folgen würden.

Aufgabe 6.6

$$\begin{aligned} \text{head}([1,2,3,4]) &= \text{head}(\text{add}([1,2,3], 4)) \\ &= \text{head}([1,2,3]) \\ &= \text{head}([1,2]) \\ &= \text{head}([1]) \\ &= \text{head}(\text{add}(\text{new}(), 1)) \\ &= 1 \end{aligned}$$

Aufgabe 6.7

$$\begin{aligned} \text{con}(\text{new}(), L) &= \text{con}(\text{new}(), \text{add}(L', e_1)) && \text{was ist mit } L = \text{new}()? \\ &= \text{add}(\text{con}(\text{new}(), L'), e_1) \\ &\vdots \\ &= \text{add}(\text{add}(\dots \text{add}(\text{con}(\text{new}(), \text{new}()), e_1, e_2, \dots, e_n) \dots e_1) \\ &= \text{add}(\text{add}(\dots \text{add}(\text{new}(), e_n, e_{n-1}, \dots, e_1) \dots e_1) \\ &= L \end{aligned}$$

warum keine Induktion über Anzahl d. Elemente?

Aufgabe 6.9

Skype:

1. Client-Server-Architektur: Findung anderer Nutzer
3. Peer-to-peer-System: Kommunikation mit anderen Nutzern

git:

7. Repositories Trifft nicht ganz zu.

Aufgabe 6.10

8. Don't Repeat Yourself

falsch:

1. function_1(n)
2. }
3. } berechne n!
4. }
5. ... (Rumpf 1)
6. function_2(n)
7. }
8. } berechne n!
9. }
10. ... (Rumpf 2)

richtig:

1. fakultaet(n)
2. }
3. } berechne n!
4. }
5. function_1(n)
6. ... (Rumpf 1)
7. function_2(n)
8. ... (Rumpf 2)

9. Self-documentation

falsch:

~~divide(a, b) {~~

do(a, b) {

return(a/b);

}

richtig:

divide(numerator, denominator) {

return(numerator/denominator);

}

1. Separation of concerns

falsch: calc(a, b, op) {
if (op == 'add') return a + b;
if (op == 'sub') return a - b;

richtig: add(a, b) { return(a + b); }
sub(a, b) { return(a - b); }
;

Aufgabe 1

$$\Sigma = (S, F)$$

$$A_{\text{int}} = \mathbb{Z}$$

$$\text{zero}_A () = 0$$

$$\text{succ}_A (n) = 1 + n$$

$$\text{inv}_A (n) = -n$$

$$\forall n \in \mathbb{Z}$$

$$\forall n \in \mathbb{Z} \setminus 0^{\text{ohne } 0}$$

Aufgabe 2

$$\text{SPEC} = (\Sigma, E) \quad \Sigma = (S, F, \chi)$$

F = Operationen

$$\text{SPEC}_{\text{list}} = \text{sorts} \quad \text{list, elem, nat}$$

vars: l: list
e: elem
l': list

ops: new: $\rightarrow \text{list}$

add: $\text{list} \times \text{elem} \rightarrow \text{list}$

del: $\text{list} \rightarrow \text{list}$

con: $\text{list} \times \text{list} \rightarrow \text{list}$

head: $\text{list} \rightarrow \text{elem}$

tail: $\text{list} \rightarrow \text{list}$

len: $\text{list} \rightarrow \text{nat}$

für die
Spezifikation
von nat

zero: $\rightarrow \text{nat}$

succ: $\text{nat} \rightarrow \text{nat}$

Constraints

$$\text{len}(\text{new}()) = \text{zero}()$$

$$\text{tail}(\text{add}(l, e)) = e$$

$$\text{len}(\text{add}(l, e)) = \text{succ}(\text{len}(l))$$

$$\text{head}(\text{add}(l, e)) = \text{head}(l)$$

$$\text{tail}(\text{del}(\text{add}(l, e))) = \text{tail}(l)$$

$$\text{len}(\text{del}(\text{add}(l, e))) = \text{len}(l)$$

$$\text{head}(\text{del}(\text{add}(l, e))) = \text{head}(l)$$

$$\text{head}(l, l') = \text{head}(l)$$

$$\text{tail}(l, l') = \text{tail}(l')$$

$$\text{head}(\text{con}(l, l')) = \text{head}(l)$$

$$\text{tail}(\text{con}(l, l')) = \text{tail}(l')$$

für nicht
leere Listen

$$\text{tail}(\text{con}(l, \text{add}(l', e))) = e$$

$$\text{con}(l, \text{new}()) = l$$